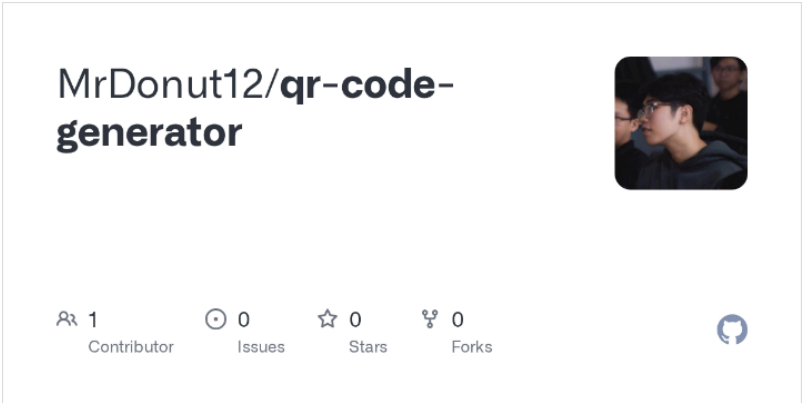
QR CODE GENERATOR API

High-Performance QR Code Generation using Java & C++ Hybrid Architecture

MrDonut12 - Overview



I. License

Overview

This project is licensed under the **MIT License**, a permissive open-source license that allows users to freely use, modify, and distribute the software with minimal restrictions.

Figure 1: *MIT License integration within the project repository*

qr-code-generator / LICENSE in main

Edit Preview Choose a license template

```
MIT License

Copyright (c) 2026 Mr.Donut

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

A short and simple permissive license with conditions only requiring preservation of copyright and license notices. Licensed works, modifications, and larger works may be distributed under different terms and without source code.

Figure 2: *Key Characteristics of MIT License*

Permissions

Commercial use

The licensed material and derivatives may be used for commercial purposes.

Modification

The licensed material may be modified.

Distribution

The licensed material may be distributed.

Private use

The licensed material may be used and modified in private.

Limitations

Liability

This license includes a limitation of liability.

Warranty

This license explicitly states that it does NOT provide any warranty.

Conditions

License and copyright notice

A copy of the license and copyright notice must be included with the licensed material.

MIT License

Copyright (c) Year Full name

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

II. Project Overview

QR-Code-Generator API is a high-performance backend service designed to generate QR codes efficiently by combining the strengths of **Java (Spring Boot)** and **C++ (native engine)**.

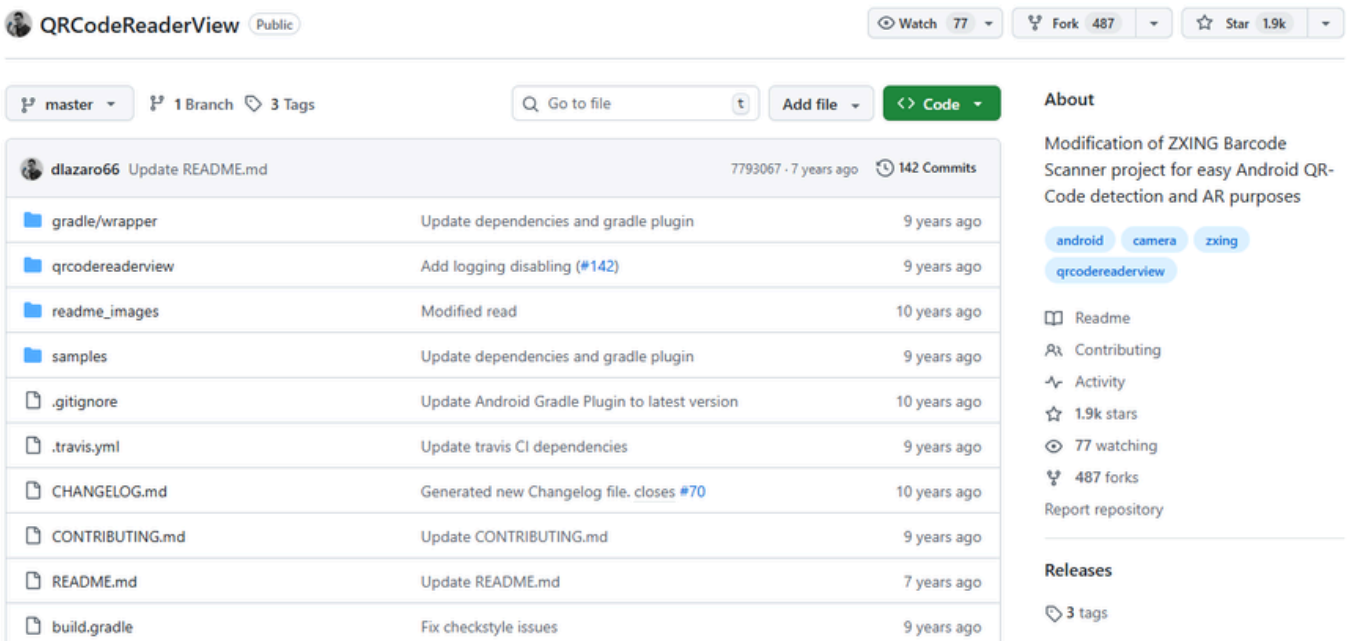
Unlike traditional QR generators that rely solely on high-level libraries, this project introduces a **hybrid architecture** where:

- **Java** handles API orchestration, scalability, and service management
- **C++** performs computationally intensive QR encoding and rendering

This design enables **low-latency processing**, **fine-grained memory control**, and **production-grade scalability**.

III. Motivation

Figure 3: *The illustration showcases the entire source code, along with contributors, forks, and stars, highlighting the project's popularity and widespread adoption.*



Most QR code generators rely on libraries such as ZXing (Java), which are convenient but:

- Abstract away low-level optimizations
- Limit control over encoding internals
- Are not optimized for high-throughput systems

This project aims to:

- Rebuild and optimize the QR encoding pipeline
- Maximize performance through native execution

- Expose a clean, scalable API interface

VI. Why Java & C++ Instead of One Language?

Why not only Java?

While Java provides excellent ecosystem support, it has **limitations:**

- Garbage Collection introduces non-deterministic latency
- Less control over memory layout and allocation
- Slower in CPU-intensive bit-level operations

Why not only C++?

C++ is powerful but:

- Lacks a robust ecosystem for rapid API development
- Requires more effort for:
  - HTTP handling
  - Dependency management
  - Deployment orchestration

Figure 4: *Deployment overview of the hybrid C++ and Java architecture, combining native performance with scalable and portable API services.*

Concern	Java (Spring Boot)	C++ (Native Engine)
API Development	*Widely used for APIs with mature frameworks (Spring Boot, Jakarta EE)*	*Not commonly used for APIs; requires more setup (e.g., frameworks like Crow, Pistache)*
Performance (CPU)	*Slightly lower due to JVM abstraction, but optimized with JIT compilation*	*Extremely high performance due to native compilation and direct memory access*
Memory Control	*Automatic (Garbage Collection) → safer and easier to manage*	*Manual (pointers, new/delete) → highly flexible but error-prone*
Concurrency	*Strong*	
Deployment	*C++ applications are compiled into native binaries that run directly on the operating system without requiring an additional runtime environment. This results in fast startup time and high efficiency. However, deployment can be more complex due to platform-specific builds and dependency management, often requiring recompilation for different environments.*	*Java applications are packaged as platform-independent artifacts (such as JAR or WAR files) and run on the Java Virtual Machine (JVM). This enables high portability across different systems. Deployment is generally simpler and more consistent, especially in cloud and enterprise environments, thanks to strong ecosystem support and integration with modern tools.*

V. Architecture

1. Architectural Overview

The system is designed using a **hybrid architecture** that separates high-level service management from low-level computational processing. The primary objective of this design is to maximize both **scalability** and **execution performance**.

The architecture consists of two main domains:

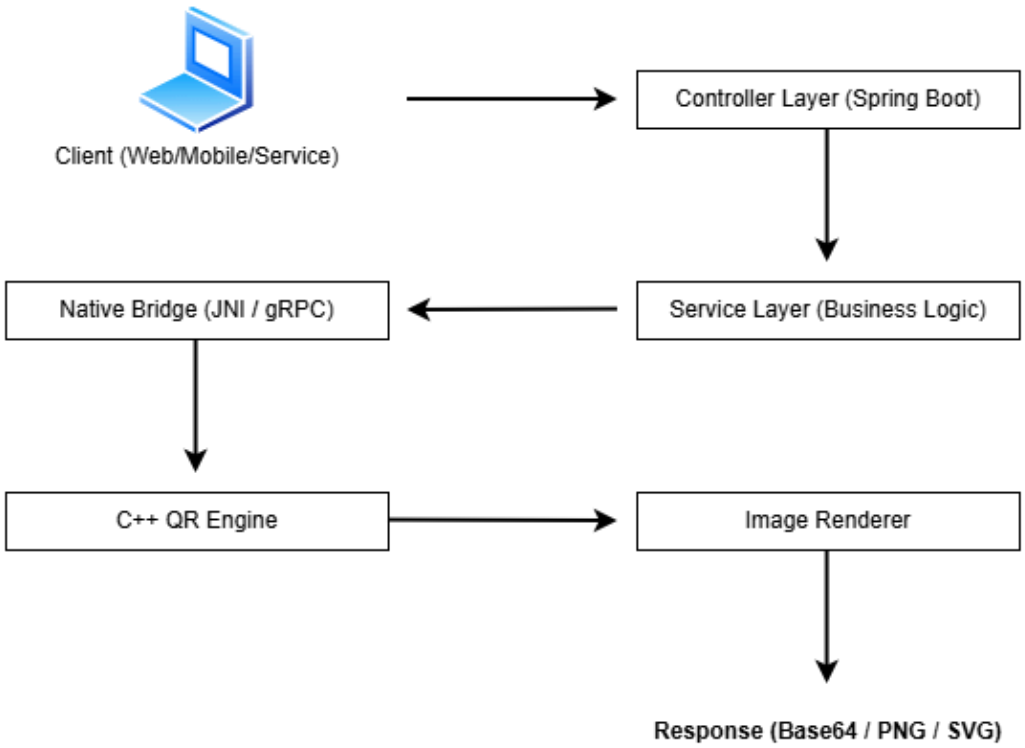
Control Plane (Java)
Responsible for managing requests, orchestration, validation, and system integration.

Data Plane (C++)
Responsible for executing the computationally intensive operations involved in QR code encoding and image generation.

This separation allows the system to scale efficiently while maintaining deterministic performance in the encoding process.

2. High-Level System Flow

Figure 5: *The lifecycle of a request within the system*



In this flow, the Java application receives the request and manages the entire lifecycle, while the C++ engine performs the performance-critical operations required to construct the QR code